

UNIVERSIDADE EDUARDO MONDLANE

FACULDADE DE ENGENHARIA

LICENCIATURA EM ENGENHARIA INFORMATICA

ESTRUTURA DE DADOS E ALGORITMOS

Tema:

Sistema de Indexação, Pesquisa e Análise de Dados com Tabela Hash

Discentes:

- Gune, Euclides
-Nogueira, Gerson Teodósio
- Saite, Cíntia de Carmen Sílvio
- Sambo, Kelvin Damião

Docentes:

-Hélio Pesanhane
- Cristiliano Maculuve

Maputo, Outubro de 2026.

Índice

1. Introdução 3
2. Objectivos 3
2.1. Objectivo geral 3
2.2. Objectivos específicos 3
3. Problema 4
4. Descrição da Solução 4
4.1. Organização do código 4
4.2. Leitura e validação do CSV 5
5. Representação dos estudantes 6
6. Organização da Tabela Hash 6
7. Funções Hash (h1 e h2) 7
7.1. Conversão da chave 7
7.2. Primeira função Hash — h1(k) 7
7.3. Segunda função Hash — h2(k) 7
7.4. Sequência de sondagem (Double Hashing) 8
8. Tratamento de colisões 8
9. Funcionamento do estado REMOVIDO 9
10. Factor de carga e redimensionamento 10
11. Estatísticas dos estudantes 10
11.1. Estatísticas gerais 11
11.2. Estatísticas por sexo 11
11.3. Estatísticas por curso 11
11.4. Estatísticas por ano 11
11.5. Estatísticas das médias 12
11.6. Estatísticas de idade 12
12. Estatísticas da Tabela Hash 12
13. Resultados experimentais 13
13.1. Metodologia 13
13.2. Resultados 14
14. Comparação entre pesquisa sequencial e Hash 14
15. Interpretação dos Resultados 15
15.1. Factor de carga e colisões 15
15.2. Colisões e sondagens 15
15.3. Sondagens e desempenho 16
15.4. Pesquisa sequencial versus Hash 16
15.5. Síntese 16
16. Conclusão 16
17. Instruções de execução 17
Referências 18

1. Introdução

A gestão de informação académica exige que os registos dos estudantes sejam armazenados e consultados de forma organizada. Cada registo contém dados como código, nome, curso, ano académico, idade, sexo e média. Quando o número de registos aumenta, a pesquisa por meio de uma verificação sequencial pode exigir a análise de muitos elementos até encontrar o estudante pretendido.

Neste contexto, a Tabela Hash constitui uma estrutura de dados adequada para associar uma chave a uma posição de armazenamento, permitindo efectuar pesquisas, inserções e remoções de forma eficiente quando a função Hash distribui as chaves de maneira satisfatória.

O presente trabalho consiste no desenvolvimento de um sistema em Java para indexação, pesquisa e análise de dados de estudantes. A solução utiliza uma Tabela Hash implementada pelo grupo e o método Double Hashing para tratar colisões. Para além de permitir a gestão dos registos, o sistema produz estatísticas e permite realizar experiências para avaliar o desempenho das operações implementadas.

2. Objectivos

2.1. Objectivo geral

Desenvolver, em Java, um sistema que leia os dados de um ficheiro CSV e os armazene numa Tabela Hash implementada pelo grupo, permitindo realizar pesquisas, inserções, remoções, estatísticas e experiências de desempenho.

2.2. Objectivos específicos

- Implementar a leitura e a validação dos registos do ficheiro estudantes.csv;
- Representar cada estudante através de um objecto com os campos definidos no enunciado;
- Implementar uma Tabela Hash baseada num array unidimensional e endereçamento aberto;
- Aplicar Double Hashing para resolver colisões;
- Disponibilizar operações de inserção, pesquisa e remoção por código;
- Implementar os estados VAZIO, OCUPADO e REMOVIDO;
- Calcular o factor de carga e realizar redimensionamento automático quando necessário;
- Produzir estatísticas dos estudantes e da própria Tabela Hash; e
- Comparar experimentalmente a pesquisa sequencial e a pesquisa Hash.

3. Problema

O problema central consiste em gerir um conjunto de milhares de registos de estudantes e permitir a sua pesquisa por código sem depender de estruturas Hash já implementadas na biblioteca padrão como estrutura principal.

A utilização de uma pesquisa sequencial torna-se menos eficiente à medida que a quantidade de registos aumenta, uma vez que, dependendo da posição do estudante, pode ser necessário percorrer muitos elementos até encontrar o resultado — ou todos, quando o código não existe.

Assim, pretende-se desenvolver uma solução que permita armazenar, pesquisar, inserir e remover estudantes de forma organizada, controlar as colisões, efectuar o redimensionamento da tabela quando necessário e analisar o comportamento da estrutura através de métricas e experiências práticas.

4. Descrição da Solução

A solução proposta consiste numa aplicação de gestão e consulta de registos académicos, desenvolvida em Java, cuja estrutura central de armazenamento é uma Tabela Hash. O sistema trabalha com o ficheiro estudantes.csv, fornecido pelos docentes, e disponibiliza operações de pesquisa, inserção, remoção, consulta estatística e avaliação de desempenho, através de um menu textual. A utilização do código do estudante como chave permite localizar um registo sem percorrer todos os elementos armazenados.

Na Tabela Hash, cada estudante é associado a uma posição do array através de uma função Hash aplicada ao seu código. Quando a posição calculada já está ocupada por outro estudante, o sistema utiliza Double Hashing para determinar posições alternativas. Cada posição mantém um estado — VAZIO, OCUPADO ou REMOVIDO — para que as pesquisas continuem a funcionar correctamente depois de remoções. Quando o factor de carga ultrapassa 0,70, a tabela é redimensionada e todos os elementos são reinseridos através das funções Hash.

Além das operações de gestão, a solução produz estatísticas sobre os dados académicos (distribuição por sexo, curso e ano, médias e idades) e sobre o comportamento da própria estrutura (capacidade, factor de carga, colisões, sondagens e redimensionamentos). Por fim, os testes experimentais com 1 000, 5 000 e 10 000 registos permitem medir o custo das operações e comparar a pesquisa sequencial com a pesquisa Hash, com códigos existentes e inexistentes.

4.1. Organização do código

O código está organizado em pacotes, cada um com uma responsabilidade:

Tabela 1 – Organização do código-fonte (pasta src)

Pacote	Classes	Responsabilidade
app	Main, Menu	Ponto de entrada e interface textual (menu).
dados	LeitorCSV, Dados	Leitura e validação do CSV; contadores da leitura.
model	Estudante, Entrada, EstadoEntrada	Representação dos estudantes e das posições da tabela.
hash	TabelaHash, CodigoDuplicadoException	Implementação da Tabela Hash com Double Hashing.
estatistica	EstatisticaGeral, EstatisticaPorSexo, EstatisticaPorCurso, EstatisticaPorAno, EstatisticaDasMedias, EstatisticaDeIdade, EstatisticaHashTable	Estatísticas dos estudantes e da Tabela Hash.
desempenho	Desempenho, PesquisaSequencial	Experiências de desempenho e pesquisa sequencial.
utils	Validacao, Resposta, EstadoValidacao, Rounder	Regras de validação e formatação de números.

4.2. Leitura e validação do CSV

A classe LeitorCSV abre o ficheiro, ignora o cabeçalho e lê os registos linha por linha. Cada linha é separada pelas vírgulas e tem de ter exactamente 7 campos; os campos ano e idade são convertidos para int e a média para double. Uma linha que não tenha 7 campos ou cujos valores numéricos não possam ser convertidos é contabilizada como inválida. Os registos convertidos são depois verificados pela classe Validacao, segundo as regras da Tabela 2.

Tabela 2 – Regras de validação de um registo

Campo	Regra
codigo	Não pode estar vazio; tem de ser único na tabela.
nome, curso	Não podem estar vazios.
ano	Inteiro entre 1 e 4.
idade	Inteiro entre 17 e 70.
sexo	M ou F (maiúscula ou minúscula).
media	Número entre 0 e 20.

Os registos válidos são inseridos na Tabela Hash. Se o código já existir, a tabela lança CodigoDuplicadoException e o registo é contabilizado à parte como duplicado (é válido, mas não é inserido). A leitura regista o total de linhas lidas, os registos válidos, os inválidos, os duplicados e os estudantes inseridos. Com o ficheiro fornecido foram lidas 10 000 linhas, todas válidas e sem códigos duplicados.

5. Representação dos estudantes

Cada estudante é representado por um objecto da classe Estudante, que reúne os seus dados académicos e pessoais. O campo codigo funciona como chave da Tabela Hash e é único.

Tabela 3 – Atributos da classe Estudante

Atributo	Tipo	Descrição
codigo	String	Código único, utilizado como chave da Tabela Hash.
nome	String	Nome completo do estudante.
curso	String	Curso frequentado pelo estudante.
ano	int	Ano académico frequentado (1 a 4).
idade	int	Idade do estudante.
sexo	String	Sexo indicado no registo (M ou F).
media	double	Média académica do estudante (0 a 20).

A utilização de uma classe para representar os estudantes permite agrupar todas as informações de cada registo num único objecto. Desta forma, a Tabela Hash armazena e devolve os dados completos do estudante a partir do seu código. Por exemplo, a pesquisa do código 20260001 (opção 2 do menu) devolve:

```
Codigo do estudante: 20260001
Estudante encontrado:
Estudante { codigo: 20260001, nome: Joao Alberto, curso: Engenharia Informatica, ano: 3, idade:
20, sexo: F, media: 7.1 }
Posicoes sondadas: 1
```

6. Organização da Tabela Hash

A Tabela Hash (classe TabelaHash) é representada por um array unidimensional de entradas, Entrada[] tabela. Cada entrada guarda uma referência para um Estudante e o estado da posição (EstadoEntrada): VAZIO, OCUPADO ou REMOVIDO. O estado permite distinguir uma posição que nunca foi utilizada de outra que continha um estudante entretanto removido.

```
class Entrada {
    private Estudante estudante;
    private EstadoEntrada estado;   // VAZIO, OCUPADO ou REMOVIDO
}
```

A capacidade inicial é de 1 009 posições. Este valor é um número primo, e a capacidade mantém-se sempre prima depois de cada redimensionamento, o que é necessário para o Double Hashing (secção 7.4). A chave é o código do estudante: a função Hash calcula uma posição inicial e, quando essa posição está ocupada por outra chave, a sequência de sondagem determina as posições seguintes a verificar. Todos os registos ficam guardados no próprio array, sem listas encadeadas.

Para respeitar os requisitos do trabalho, a estrutura principal foi implementada pelo próprio grupo, sem utilizar HashMap, Hashtable, HashSet, LinkedHashMap ou TreeMap. A implementação disponibiliza as operações obrigatórias void inserir(Estudante estudante), Estudante pesquisar(String codigo) e boolean remover(String codigo), bem como os métodos que fornecem as estatísticas da estrutura.

7. Funções Hash (h1 e h2)

A implementação utiliza duas funções Hash, h1 e h2, que trabalham em conjunto para determinar as posições onde os estudantes são armazenados e pesquisados. Como o código do estudante é do tipo String, é primeiro convertido num número inteiro k.

7.1. Conversão da chave

```
private int converterChave(String codigo) {
    int k = 0;
    for (int i = 0; i < codigo.length(); i++) {
        k = 31 * k + codigo.charAt(i);
    }
    return k & 0x7FFFFFFF;
}
```

Cada carácter do código contribui para k através do polinómio k = 31·k + carácter, de forma que códigos diferentes produzem, em geral, valores diferentes. Os valores podem ultrapassar o limite de um int e ficar negativos; a operação & 0x7FFFFFFF anula o bit de sinal e garante que k nunca é negativo (é mais segura do que Math.abs, que falha para Integer.MIN_VALUE). Para o código 20260001 obtém-se k = 226 996 227.

7.2. Primeira função Hash — h1(k)

A primeira função determina a posição inicial do estudante na tabela:

```
private int h1(int k) {
    return k % tabela.length;
}
```

$$h_1(k) = k \bmod m$$

em que k é o valor inteiro obtido a partir do código e m é a capacidade actual da tabela (tabela.length). O resto da divisão está sempre entre 0 e m − 1, logo é uma posição válida do array. Com m = 1 009, o código 20260001 tem h1 = 226 996 227 mod 1 009 = 488.

7.3. Segunda função Hash — h2(k)

A segunda função determina o passo usado para avançar na tabela quando ocorre uma colisão:

```
private int h2(int k) {
    return 1 + (k % (tabela.length - 1));
}
```

$$h_2(k) = 1 + \left(k \bmod (m-1)\right)$$

O resultado está sempre entre 1 e m − 1. Nunca é zero: um passo igual a zero faria a sondagem ficar sempre na mesma posição. Com m = 1 009, o código 20260001 tem h2 = 1 + (226 996 227 mod 1 008) = 676.

7.4. Sequência de sondagem (Double Hashing)

As duas funções são combinadas para calcular a posição examinada em cada tentativa i (i = 0, 1, 2, …):

$$h(k,i) = \left(h_1(k) + i \cdot h_2(k)\right) \bmod m$$

Na tentativa i = 0 examina-se a posição h1(k). Se essa posição estiver ocupada por outro estudante, avança-se h2(k) posições (voltando ao início do array quando se ultrapassa o fim), e assim sucessivamente, até encontrar uma posição livre ou o código procurado. Como o passo depende da chave, duas chaves com a mesma posição inicial seguem, em geral, caminhos diferentes, o que evita a formação de grandes blocos de posições ocupadas.

Como m é primo e h2(k) está entre 1 e m − 1, o passo e a capacidade não têm divisores comuns. Por isso, a sequência de sondagem visita todas as m posições da tabela antes de se repetir: se existir uma posição livre, ela é sempre encontrada.

8. Tratamento de colisões

Uma colisão ocorre quando a posição examinada durante uma inserção já está ocupada por outro estudante — por exemplo, quando dois códigos diferentes têm a mesma posição inicial h1. Nesse caso o algoritmo continua a sequência de sondagem até encontrar uma posição livre. Na implementação contam-se:

- sondagem: cada posição examinada durante uma inserção;
- colisão: cada posição examinada que está OCUPADA por outro estudante (as posições REMOVIDO não contam, porque podem ser reutilizadas).

Sem remoções, cada inserção faz exactamente colisões + 1 sondagens; por isso, com os 10 000 registos, o total de sondagens (20 434) é igual ao total de colisões (10 434) mais o número de inserções (10 000).

Se durante a sondagem for encontrado um estudante com o mesmo código, a inserção é recusada com CodigoDuplicadoException. Na leitura do CSV esse registo é contado como duplicado; no menu é apresentada a mensagem «Ja existe um estudante com esse codigo.».

Exemplo real, obtido com a opção 9 do menu depois de carregar os 10 000 registos (m = 16 249): o código 20267732 tem a mesma posição inicial (1818) que o código 20261739, que já lá estava. A pesquisa de 20267732 colide quatro vezes antes de o encontrar:

```
Código 20267732: k=227211585, h1=1818, h2=15802, m=16249
  i=0 -> posição 1818: OCUPADO (20261739) <- colisão
  i=1 -> posição 1371: OCUPADO (20265100) <- colisão
  i=2 -> posição 924: OCUPADO (20267657) <- colisão
  i=3 -> posição 477: OCUPADO (20265025) <- colisão
  i=4 -> posição 30: OCUPADO (20267732) <- encontrado
```

9. Funcionamento do estado REMOVIDO

Ao remover um estudante, a sua posição não é marcada como VAZIO, mas sim como REMOVIDO. Se fosse marcada como VAZIO, os estudantes que colidiram com ela e foram colocados mais à frente na sequência de sondagem deixariam de ser encontrados, porque a pesquisa pára na primeira posição VAZIO. A Tabela 4 resume o comportamento de cada operação perante os três estados.

Tabela 4 – Comportamento das operações perante cada estado

Estado	Pesquisa	Inserção
VAZIO	Pára: o código não existe.	Pára: o estudante é colocado aqui (ou no primeiro REMOVIDO encontrado antes).
OCUPADO	Se for o código procurado, termina; senão continua.	Se for o mesmo código, é duplicado; senão é uma colisão e continua.
REMOVIDO	Continua a sondar.	Guarda a primeira posição REMOVIDO para a reutilizar, mas continua até VAZIO para confirmar que o código não existe mais à frente.

A inserção não pára no primeiro REMOVIDO porque o mesmo código pode estar mais à frente na sequência; só depois de chegar a uma posição VAZIO é que o estudante é colocado no primeiro REMOVIDO encontrado, reaproveitando-o.

As posições REMOVIDO não contam para o factor de carga, mas também não são VAZIO. Para evitar que, após muitas remoções, as pesquisas de códigos inexistentes percorram uma grande parte da tabela, quando (ocupados + removidos) / m ultrapassa 0,70 a tabela é reconstruída com a mesma capacidade, o que elimina as posições REMOVIDO.

Teste de compreensão (secção 18 do enunciado), executado com o exemplo da secção anterior: depois de remover 20261739 (opção 4), a pesquisa de 20267732 encontra a posição inicial marcada como REMOVIDO e continua até encontrar o estudante:

```
Código 20267732: k=227211585, h1=1818, h2=15802, m=16249
  i=0 -> posição 1818: REMOVIDO
  i=1 -> posição 1371: OCUPADO (20265100) <- colisão
  i=2 -> posição 924: OCUPADO (20267657) <- colisão
  i=3 -> posição 477: OCUPADO (20265025) <- colisão
  i=4 -> posição 30: OCUPADO (20267732) <- encontrado
```

10. Factor de carga e redimensionamento

O factor de carga indica a proporção de posições ocupadas na tabela:

$$\alpha = \frac{n}{m}$$

em que n é o número de posições OCUPADAS e m a capacidade da tabela. Depois de cada inserção, se α > 0,70, a tabela é redimensionada:

- a nova capacidade é o primeiro número primo maior ou igual a 2m;
- todos os estudantes são reinseridos na nova tabela através de h1 e h2, calculadas com a nova capacidade. Não são copiados para as mesmas posições do array antigo, porque com outro m as posições mudam;
- as posições REMOVIDO não são copiadas, pelo que desaparecem.

A comparação com 0,70 usa o valor exacto de α, sem arredondamento. Com os 10 000 registos do ficheiro ocorreram 4 redimensionamentos:

Tabela 5 – Redimensionamentos durante a leitura dos 10 000 registos

N.º	Inserção n.º	α antes	Capacidade	α depois
1	707	0,70069	1 009 → 2 027	0,34879
2	1 419	0,70005	2 027 → 4 057	0,34977
3	2 840	0,70002	4 057 → 8 117	0,34988
4	5 682	0,70001	8 117 → 16 249	0,34968

Cada redimensionamento reduz o factor de carga para cerca de 0,35. Assim, as inserções acontecem sempre com α entre 0,35 e 0,70, o que limita o número de colisões e de sondagens (ver secção 15).

11. Estatísticas dos estudantes

Após a leitura do CSV (opção 1), o programa apresenta automaticamente o relatório estatístico, que pode ser consultado novamente na opção 5. As estatísticas são calculadas sobre os estudantes que estão na Tabela Hash, pelo que reflectem também as inserções e remoções feitas pelo menu. Os valores seguintes correspondem aos 10 000 registos do ficheiro.

11.1. Estatísticas gerais

Tabela 6 – Estatísticas gerais da leitura

Indicador	Valor
Total de linhas lidas	10 000
Registos válidos	10 000
Registos inválidos	0
Registos duplicados	0
Estudantes inseridos	10 000

11.2. Estatísticas por sexo

Tabela 7 – Estudantes por sexo

Sexo	Quantidade	Percentagem
Masculino	4 948	49,48%
Feminino	5 052	50,52%

11.3. Estatísticas por curso

Tabela 8 – Estudantes por curso

Curso	Estudantes	Percentagem	Média académica
Engenharia Electronica	2 505	25,05%	12,53
Engenharia Informatica	2 460	24,60%	12,42
Engenharia Telecomunicacoes	2 433	24,33%	12,40
Engenharia de Redes	2 602	26,02%	12,41

11.4. Estatísticas por ano

Tabela 9 – Estudantes por ano

Ano	Estudantes	Percentagem
1.º ano	2 482	24,82%
2.º ano	2 510	25,10%
3.º ano	2 482	24,82%
4.º ano	2 526	25,26%

11.5. Estatísticas das médias

Tabela 10 – Estatísticas das médias académicas

Indicador	Valor
Média académica geral	12,44
Maior média	20,0 — Carlos Jose (20260328) e mais 28 estudantes
Menor média	5,0 — Samuel Ernesto (20268896) e mais 42 estudantes
Média inferior a 10	3 336 (33,36%)
Média entre 10 e 13,9	2 679 (26,79%)
Média entre 14 e 16,9	2 004 (20,04%)
Média igual ou superior a 17	1 981 (19,81%)

Como vários estudantes partilham a maior e a menor média, o programa indica o primeiro encontrado e quantos outros têm o mesmo valor.

11.6. Estatísticas de idade

Tabela 11 – Estatísticas de idade

Indicador	Valor
Idade média	23,45 anos
Menor idade	17 anos
Maior idade	30 anos
Menos de 18 anos	750 (7,50%)
18 a 20 anos	2 118 (21,18%)
21 a 23 anos	2 183 (21,83%)
24 a 26 anos	2 088 (20,88%)
Mais de 26 anos	2 861 (28,61%)

Os estudantes estão distribuídos de forma equilibrada por sexo (cerca de 50% cada), por curso (entre 24,3% e 26,0%) e por ano (cerca de 25% cada). As médias por curso são muito próximas (12,40 a 12,53). Um terço dos estudantes (33,36%) tem média inferior a 10 e cerca de um quinto (19,81%) tem média igual ou superior a 17. O intervalo etário com mais estudantes é o de mais de 26 anos (28,61%), que abrange quatro idades (27 a 30), enquanto os outros intervalos abrangem três.

12. Estatísticas da Tabela Hash

A opção 6 do menu apresenta as métricas da própria estrutura. Depois de carregar os 10 000 registos:

Tabela 12 – Estatísticas da Tabela Hash após a leitura do CSV

Métrica	Valor
Capacidade inicial	1 009
Capacidade final	16 249
Elementos armazenados	10 000
Factor de carga final	0,6154
Total de colisões	10 434
Total de sondagens	20 434
Média de sondagens por inserção	2,04
Maior número de sondagens	22
Número de redimensionamentos	4

Em média, cada inserção examinou cerca de duas posições, ou seja, cerca de uma colisão por inserção. A inserção mais cara examinou 22 posições, valor muito inferior aos 10 000 elementos armazenados. O factor de carga final (0,6154) está abaixo do limite de 0,70, porque o último redimensionamento ocorreu na inserção n.º 5 682.

13. Resultados experimentais

13.1. Metodologia

As experiências são executadas pela opção 7 do menu (classe Desempenho), com os primeiros 1 000, 5 000 e 10 000 registos do ficheiro. Para que os tempos medidos correspondam apenas às operações da Tabela Hash:

- o ficheiro é lido uma só vez, antes das medições; o tempo de inserção mede apenas a inserção dos estudantes numa tabela nova (incluindo os redimensionamentos), sem a leitura do disco;
- a pesquisa é medida com 1 000 códigos existentes, distribuídos uniformemente pelo ficheiro (início, meio e fim), e apresenta-se o tempo médio por pesquisa;
- os tempos são medidos com System.nanoTime(). Antes de medir, cada operação é executada algumas vezes para aquecimento, porque a máquina virtual Java compila o código durante as primeiras execuções; depois, cada medição é repetida 10 vezes e usa-se a mediana, para que uma repetição isolada mais lenta (por exemplo, devido à recolha de lixo) não distorça o resultado.

Os testes foram executados num computador com processador Intel Pentium Silver N5030 (1,10 GHz), 4 GB de memória RAM, Windows 11 e Java 26. Os tempos dependem do computador e variam ligeiramente entre execuções; as colisões, as sondagens e o factor de carga

são sempre os mesmos para o mesmo ficheiro. A saída completa está no ficheiro resultados/resultados_experimentais.txt, entregue com o código.

13.2. Resultados

Tabela 13 – Resultados da experiência de desempenho

Registos	Colisões	Média sond.	Máx. sond.	Carga	Tempo inserção	Tempo pesquisa
1 000	615	1,62	11	0,4933	0,119 ms	55,1 ns
5 000	4 446	1,89	15	0,6160	1,168 ms	73,6 ns
10 000	10 434	2,04	22	0,6154	2,552 ms	98,5 ns

O tempo de inserção é o tempo total para inserir todos os registos; o tempo de pesquisa é o tempo médio de uma pesquisa de um código existente. Com 1 000 registos a tabela terminou com capacidade 2 027 (1 redimensionamento); com 5 000, com capacidade 8 117 (3 redimensionamentos); e com 10 000, com capacidade 16 249 (4 redimensionamentos).

14. Comparação entre pesquisa sequencial e Hash

A pesquisa sequencial (classe PesquisaSequencial) percorre os estudantes pela ordem do ficheiro, um a um, até encontrar o código ou chegar ao fim. No caso médio, um código existente exige cerca de n/2 comparações e um código inexistente exige n comparações, pelo que a sua complexidade é O(n). A pesquisa Hash tem custo médio O(1) quando o factor de carga é controlado; no pior caso, com muitas colisões, pode aproximar-se de O(n).

A comparação é executada pela opção 8 do menu. Para cada quantidade de registos, os dois métodos pesquisam exactamente os mesmos códigos: 1 000 códigos existentes, distribuídos pelo ficheiro, e 1 000 códigos inexistentes (99990000 a 99990999). A metodologia de medição é a descrita na secção 13.1.

Tabela 14 – Tempo médio por pesquisa: sequencial vs Hash

Registos	Códigos	Hash	Sondagens	Sequencial	Comparações	Sequencial / Hash
1 000	existentes	52,4 ns	1,15	5 772 ns	501	110×
1 000	inexistentes	48,6 ns	1,45	9 337 ns	1 000	192×
5 000	existentes	58,3 ns	1,25	28 137 ns	2 499	483×
5 000	inexistentes	82,4 ns	2,64	57 714 ns	5 000	700×
10 000	existentes	92,6 ns	1,79	107 038 ns	4 996	1 157×
10 000	inexistentes	97,2 ns	2,24	135 273 ns	10 000	1 392×

Sondagens: número médio de posições da tabela examinadas por pesquisa. Comparações: número médio de estudantes comparados pela pesquisa sequencial.

15. Interpretação dos Resultados

Os resultados confirmam a relação Factor de carga → Colisões → Sondagens → Desempenho.

15.1. Factor de carga e colisões

Quanto maior o factor de carga, maior a probabilidade de a posição calculada já estar ocupada e, portanto, de ocorrer uma colisão. O redimensionamento impede que α ultrapasse 0,70: sempre que esse limite é ultrapassado, α desce para cerca de 0,35 (Tabela 5). Por isso, o número de colisões cresce de forma aproximadamente proporcional ao número de registos — cerca de 0,6 colisões por inserção com 1 000 registos e cerca de 1 com 10 000 — em vez de aumentar cada vez mais à medida que a tabela enche. Com 1 000 registos há menos colisões por inserção porque a experiência termina com $\alpha = 0{,}49$, e as últimas inserções foram feitas numa tabela pouco ocupada.

Os códigos do ficheiro são números consecutivos (20260001, 20260002, …). Como a conversão $k = 31 \cdot k +$ carácter é linear, códigos que só diferem no último dígito dão valores de k que diferem em 1 e, portanto, posições iniciais h1 vizinhas e distintas. Isto ajuda a espalhar as chaves pela tabela, sobretudo quando há poucos registos.

15.2. Colisões e sondagens

Cada colisão obriga a examinar mais uma posição, pelo que as sondagens acompanham as colisões. A média de sondagens por inserção estabiliza perto de 2 (1,62, 1,89 e 2,04) e o número máximo cresce apenas de 11 para 22, enquanto o número de registos aumenta dez vezes.

Nas pesquisas, o número de sondagens é próximo do previsto pela teoria para endereçamento aberto (Cormen et al., 2022): para $\alpha \approx 0{,}62$, uma pesquisa com sucesso deve examinar em média cerca de $(1/\alpha)\cdot\ln(1/(1-\alpha)) \approx 1{,}55$ posições, e uma pesquisa sem sucesso cerca de $1/(1-\alpha) \approx 2{,}6$ posições. Os valores medidos com 5 000 e 10 000 registos foram 1,25 e 1,79 para códigos existentes e 2,64 e 2,24 para códigos inexistentes, próximos dos teóricos. Com 1 000 registos ($\alpha \approx 0{,}49$) a teoria prevê 1,38 e 1,97, e mediram-se valores mais baixos (1,15 e 1,45), o que é coerente com a boa distribuição dos códigos consecutivos. Os códigos inexistentes exigem mais sondagens, porque a pesquisa só termina ao encontrar uma posição VAZIO.

As pesquisas de códigos existentes examinam menos posições (1,15 a 1,79) do que a média registada durante as inserções (1,62 a 2,04). Isto acontece porque os redimensionamentos reinserem os estudantes numa tabela maior e menos ocupada, onde muitos deles passam a ficar na sua posição inicial.

15.3. Sondagens e desempenho

Como as sondagens por pesquisa se mantêm entre 1 e 3, o tempo de pesquisa Hash quase não depende do número de registos: com dez vezes mais registos, o tempo médio passou de cerca de 50 ns para cerca de 95 ns. Este aumento é pequeno e deve-se ao ligeiro aumento das sondagens e, provavelmente, ao facto de uma tabela maior ocupar mais memória, tornando os acessos menos favoráveis à memória cache do processador. Este comportamento corresponde ao custo médio O(1).

O tempo de inserção cresce aproximadamente de forma proporcional ao número de registos (0,119 ms, 1,168 ms e 2,552 ms). Embora cada redimensionamento reinsira todos os elementos, a capacidade duplica de cada vez, pelo que o custo dos redimensionamentos, distribuído por todas as inserções, se mantém constante por inserção.

15.4. Pesquisa sequencial versus Hash

O tempo da pesquisa sequencial cresce com o número de registos, como previsto para O(n): o número médio de comparações foi praticamente n/2 para códigos existentes (501, 2 499 e 4 996) e exactamente n para códigos inexistentes. Com 10 000 registos, uma pesquisa sequencial demorou em média cerca de 107 µs (código existente) e 135 µs (código inexistente), contra menos de 0,1 µs na Tabela Hash.

A vantagem da Tabela Hash aumenta com o número de registos: foi cerca de 110 a 190 vezes mais rápida com 1 000 registos e 1 157 a 1 392 vezes mais rápida com 10 000. O caso mais favorável à Tabela Hash é o dos códigos inexistentes, em que a pesquisa sequencial percorre todos os registos, enquanto a pesquisa Hash pára na primeira posição VAZIO.

15.5. Síntese

Manter o factor de carga abaixo de 0,70 limita as colisões; poucas colisões significam poucas sondagens por operação; e poucas sondagens tornam o custo das operações praticamente independente do número de registos. Os tempos absolutos dependem do computador utilizado, mas as conclusões mantêm-se, porque se baseiam também nas contagens de colisões, sondagens e comparações, que não dependem da máquina.

16. Conclusão

Com a realização deste trabalho, foi desenvolvido um sistema de gestão de estudantes em Java, com uma Tabela Hash implementada pelo grupo, com Double Hashing. O sistema carrega e

valida os dados do ficheiro CSV, permite pesquisar, inserir e remover estudantes, calcula estatísticas dos estudantes e da tabela e mede o desempenho das operações.

A implementação permitiu compreender a importância das funções de dispersão, do tratamento de colisões, do estado REMOVIDO e do redimensionamento. Os testes mostraram que, mantendo o factor de carga abaixo de 0,70, cada operação examina em média menos de três posições, mesmo com 10 000 registos. Por isso, a pesquisa Hash foi mais de mil vezes mais rápida do que a pesquisa sequencial com 10 000 registos, e esta vantagem aumenta com o número de registos.

Desta forma, o trabalho relacionou a teoria — O(1) no caso médio para a Tabela Hash e O(n) para a pesquisa sequencial — com resultados medidos, e mostrou a importância de escolher estruturas de dados adequadas para a pesquisa e gestão eficiente de dados.

17. Instruções de execução

O programa requer o JDK 17 ou superior e não usa bibliotecas externas. Os comandos devem ser executados na pasta raiz do projecto (a que contém a pasta src), porque o ficheiro é lido de src/dados/estudantes.csv.

Compilar e executar no Windows (PowerShell):

```
javac -encoding UTF-8 -d out (Get-ChildItem -Recurse -Filter *.java src).FullName
java -cp out app.Main
```

Compilar e executar em Linux, macOS ou Git Bash:

```
javac -encoding UTF-8 -d out $(find src -name "*.java")
java -cp out app.Main
```

Também é possível abrir a pasta do projecto no VS Code ou no IntelliJ e executar a classe app.Main. O menu da aplicação é o seguinte:

```
==============================================
       SISTEMA DE GESTAO DE ESTUDANTES
==============================================
1 - Carregar ficheiro CSV
2 - Pesquisar estudante
3 - Inserir estudante
4 - Remover estudante
5 - Estatisticas dos estudantes
6 - Estatisticas da Hash Table
7 - Executar teste de desempenho
8 - Comparar pesquisa sequencial e Hash
9 - Demonstrar colisao e REMOVIDO (sondagem)
0 - Sair
==============================================
```

As opções 2 a 6 e 9 exigem que o ficheiro tenha sido carregado (opção 1). As opções 7 e 8 lêem o ficheiro por conta própria e demoram alguns segundos. A opção 9 indica um par de códigos

em colisão e mostra a sequência de sondagem de um código, o que permite realizar o teste de compreensão descrito na secção 9.

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 4.ª ed. Cambridge (MA): MIT Press, 2022. Capítulo 11 – Hash Tables.

SEDGEWICK, R.; WAYNE, K. Algorithms. 4.ª ed. Boston: Addison-Wesley, 2011. Secção 3.4 – Hash Tables.

MACULUVE, C.; PESANHANE, H. Trabalho Prático de Estruturas de Dados e Algoritmos: Sistema de Indexação, Pesquisa e Análise de Dados com Tabela Hash. Maputo: Universidade Eduardo Mondlane, Faculdade de Engenharia, 2026.